



Nicolas Jimenez-Lozano, Pat Mutia, Athena Kolli

ME/CS/EE 133b: Robotics

Bloxorz

March 2024

Final Project

Contents

1	Introduction	1
1.1	Motivation	1
1.1.1	Game Overview	1
1.1.2	Code Structure	2
2	The Task	3
2.0.1	Version 1: Dijkstra's on a basic map	3
2.0.2	Version 2: A-star on a basic map	4
2.0.3	Version 3: A-star with buttons	4
3	Cost Metric Design	5
3.0.1	Cost Metric Considerations	5
3.0.2	Attempted Solutions	6
4	Conclusion	10
4.0.1	Conclusion	10
4.0.2	Project Takeaways	10
4.0.3	Special Thanks	11
4.0.4	Link to Video	11

Chapter 1

Introduction

1.1 Motivation

This project sought to advance our comprehension of robotics and path planning by delving deeper into a very familiar topic but with a more complex problem and analysis. Our focus centered on running grid searches, specifically on exploring cost metrics. We were intrigued by problems such as Sokoban and the Rook, which had no good cost metric in the case of Sokoban, or cost metrics that were not directly correlated to distance from the goal, in the case of the Rook. This is an interesting problem to tackle as useful cost metrics can significantly reduce run time and map exploration. We chose Bloxorz as our vehicle for this analysis in hope that we would be able to find cost metrics within the framework of a game, which requires general movement towards a goal but sometimes backtracking and general movements away from a goal.

1.1.1 Game Overview

Bloxorz is a game where the user can play as a 2×1 rectangular prism and has an objective of getting from its initialized position to a 1×1 hole. The user plays on a grid that can be variably arranged to constrict the moves that the user can perform, and moreover the grid can have additional features like buttons and tiles that cannot be stood, which adds additional complexity. Starting in an standing orientation, the block move in the four directions: *RIGHT*, *UP*, *LEFT*, *DOWN* and be characterized in states of being in a standing orientation, a laying horizontal orientation, and a laying vertical orientation. Therefore, the start and goal orientation are a standing orientation.

1.1.2 Code Structure

Bloxorz has multiple game elements that would produce complex planning problem such as different button types that control bridges, X's that open bridges but must have a specific orientation to active that button, and parentheses buttons that separate the 2×1 into two 1×1 . Therefore, we decided to tackle the button that would access bridge tiles as a relatively complex component to test the planning algorithms on. We defined five classes: **Direction**, **Block**, **Space**, **Maze**, **Node**. For directions were the regular **RIGHT**, **UP**, **LEFT**, **DOWN**, but additionally we defined deltas in the X and Y direction for when the block was standing, laying horizontally, or laying vertically then was commanded to move in those directions specifically. We defined the **Block** to have a **center** and **orientation**, which is also just summarized in **state** as a tuple. For **Block**, we can look at the state of the **Block** with adjacent moves. **Space** are used to defined the tiles in the grid, so they have a **r** for row, **c** for column, and **type** that can be **tile**, **button**, **hidden-unpressed**, **hidden-pressed**. The **hidden-unpressed** and **hidden-pressed** represent the bridge tiles that are activated using the button that can be activated or deactivated more than once. Then **Maze** uses the **Space** objects to define the grid and the visuals for the grid, which code structure we referenced from the Sokoban assignment. For the **Node** class, we use the nodes for planning and seeing where we have gone and cannot go. Therefore, there are functions that check when the button is pressed, transition the block while checking if its legal to perform that move, getting children for the planner, and distance checkers for the cost metric. We initially used Dijkstra for our basic maps that worked relatively well, but we noticed A-Star was comparable or better for directing the block towards the goal. However, the cases were more complex with the buttons and configurations of the map when the grid had spaces near the goal but were blocked off.

Chapter 2

The Task

We implemented various cost metrics and discrete planners in order to solve various challenges

2.0.1 Version 1: Dijkstra's on a basic map

Version 1 of the Bloxorz game solves the problem of getting from a starting position to a goal without an additional game components (i.e. no buttons). This simplified version of the game only requires a block position and orientation in order to define the current state and operated quite rapidly.

For Dijkstra, we know the cost-to-reach to any trunk node y (**done**) is bound by the cost-to-reach of any leaf node x (**ondeck**). Then the cost-to-reach any trunk node y (**done**) is optimal. When considering a new path to any node y , this path will have to pass through a leaf node. So, all undiscovered paths will show a cost-to-reach at least as high as the lowest cost-to-reach of any leaf. Therefore, the leaf with minimal cost-to-reach has no better path and can be transferred to the trunk, meaning we found a path with minimal cost from the start. For Bloxorz, we defined the cost as the distance metric of Manhattan and Chebyshev, which perform equivalently for simple maps.

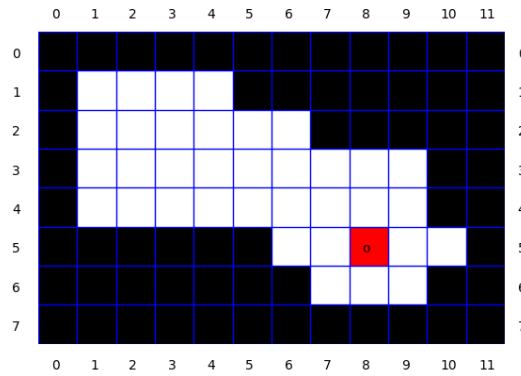


Figure 2.1: Level 1 of Bloxorz, found with 7 steps

2.0.2 Version 2: A-star on a basic map

Version 2 of the Bloxorz game solves the same problem as version 1, but now implements A-star in order to get cost metrics to guide the block towards the goal thus reducing the overall exploration of the block. The primary cost metrics we tried were Manhattan and Chebyshev.

2.0.3 Version 3: A-star with buttons

Version 3 attempts to solve a more complex version of the problem. With a more complicated game state (position, orientation, and button state), the number of nodes increases dramatically. This can be seen in the two cases below. On the same map but with one button position moved, the number of explored nodes goes from 964 to 10817. This validates the main motivation for the exploring the cost metrics.

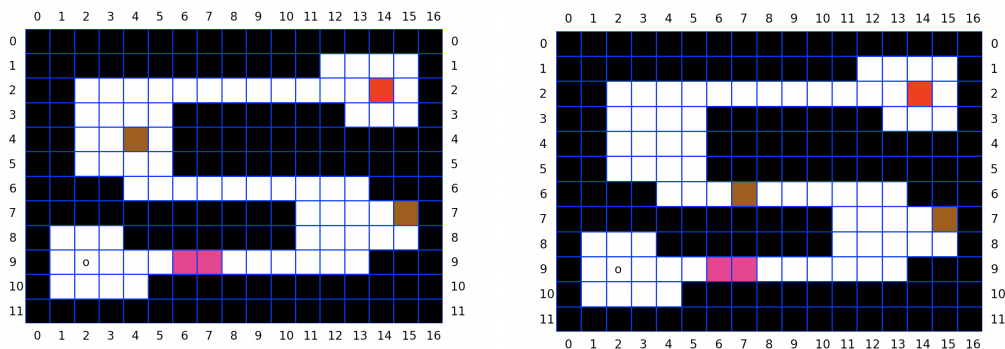


Figure 2.2: Similar maps with drastically different run times and number of nodes explored

Chapter 3

Cost Metric Design

In this section, we will explore the paths achieved by star with a particular focus on optimality and number of visited nodes.

3.0.1 Cost Metric Considerations

Introduction to the Problems

This problem of finding useful cost metrics appears simple initially, yet the subtle intricacies of this problem make this problem difficult to solve. Since only blocks with the **UP** orientation (1x1 footprint) can fit in the goal, being on top of the goal with a **VERTICAL** (2x1 footprint) or a **HORIZONTAL** orientation (1x2 footprint) which would generally give a cost-to-go of 0 actually represent a non-zero cost left. This is similar to the knight chess problem, where proximity to the goal does not necessarily scale with steps needed to reach the goal. Buttons to unlock hidden tiles also complicate this problem, as they are usually necessary to solve the problem, but are ignored by typical cost metrics.

One Tile Away

An interesting problem is the difference in cost metric utility depending on distance to the goal. The number of steps left generally scales with distance, with one main exception: one-tile-away cases:

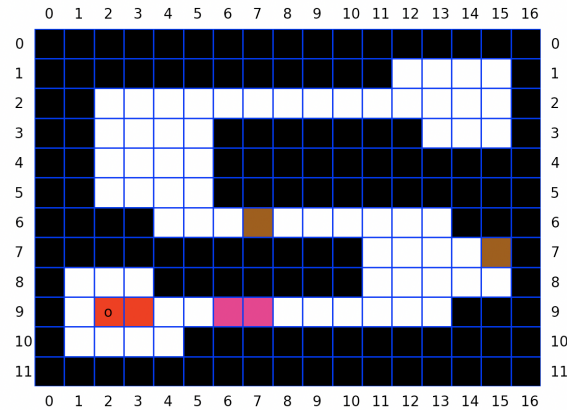


Figure 3.1: Position with perceived distance of 0.5, but which requires 8 moves to resolve

This means that cost metrics will not take into account that getting near the goal may actually force additional steps.

Buttons

Buttons also complicate cost metric design. A good cost metric should account for the existence and state of buttons in some way or another. In most maps with buttons, the buttons tend to be out of the optimal path if the path were not blocked by hidden tiles. This means that traditional cost metrics will penalize the action of pressing a button even if it is necessary to get to the goal.

3.0.2 Attempted Solutions

Cost-Switching Manhattan

The proposed modification is as follows:

- Assume the button is a crucial game element
- If the current state does not have the button pressed, cost-to-go is the cost to go to the nearest button
- If the button has been pressed, cost-to-go is the cost to go to the goal

Accounting for the button with cost-switching did not provide any noticeable improvement, instead it tripled runtime and doubled the number of visited nodes. This is due to the transition between the two costs. The path is directed towards the button for the initial portion, but once the button is pressed these nodes near the button which had a really low cost-to-go before suddenly jump up. This means that the nodes closer to the start are then prioritized and we essentially begin from 0. This is easy to improve, though, which leads to the following type:

cost-switching with button priortization manhattan

The proposed modification is as follows:

- Assume the button is a crucial game element
- If the current state does not have the button pressed, cost-to-go is the cost to go to the nearest button
- If the button has been pressed, cost-to-go is the cost to go to the goal
- Order On-deck by cost, but placing all states with the button pressed before those without it being pressed.

This solution resolves the issue from the cost-switching method and provides a significantly improvement both on runtime and DONE nodes. This method is also consistent with the logic being as follows: If we assume all paths to the goal must pass through a button then it must be true that ordering the On-deck by button pressing will result in an optimal path since all optimal paths must have the button pressed. And since manhattan distance is a consistent metric, we know that the path will be optimal

Broken-up manhattan

The proposed modification is as follows:

- Assume the button is a crucial game element
- For every button on the board. Chain two Astars. One from the start to the button and one from the button to the goal

The broken-up method showed extraordinary improvement over the other case in terms of runtime and visited nodes. The main issue with this method is that the result it gave was not

optimal. This is intuitively true, as we can't expect the sum of two optimal paths (A to B and B to C) to be the optimal path from A to C. In our case, the reason is pretty simple, the first state to reach the button is chosen as the starting point for the next Astar, this first state, however, might not put it in the best situation to reach the goal.

Manhattan + Wall Repelling

The method involved adding an additional cost for states that have walls adjacent to the current position (r, c) of the block by checking how many valid tiles there are adjacent to the block (*RIGHT*, *UP*, *LEFT*, *DOWN*) and subtracting that number from four to get the number of adjacent walls. The number of walls are then multiplied by a factor and added to the cost to go, to effectively deprioritize being near walls. This pushed the path more internal to the map, but was not necessarily optimal.

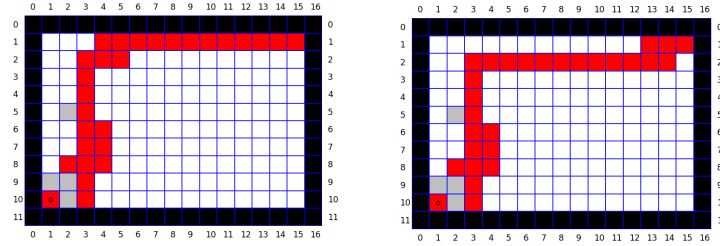


Figure 3.2: Exploration map left: Regular Manhattan, right: Manhattan with Wall Repelling

Manhattan + Wall Repelling + Near-field Button Prioritization

In addition to the added cost measures in the previous section, this strategy involved checking if any adjacent tile was a button and heavily reducing the cost towards this node, to encourage any buttons to be pressed.

Manhattan Comparison

These proposed methods were tested on a fictitious map without any obstacles. This grid was chosen because a large number of possible orientations would highlight the benefit of a star and having an important button far from the goal would show the benefit of button consideration.

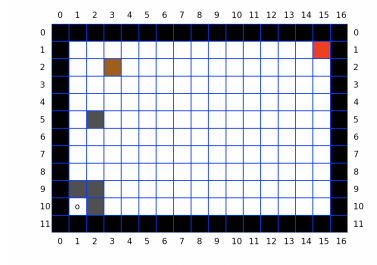


Figure 3.3: Initial state

Type	total Done nodes	run time	total path-length
Manhattan	7410	41	16
cost-switching	14131	121.73	16
cost-switching with button prioritization	1213	5.47	16
broken-up manhattan	72	0.42	18

Table 3.1: Analysis of various cost metrics

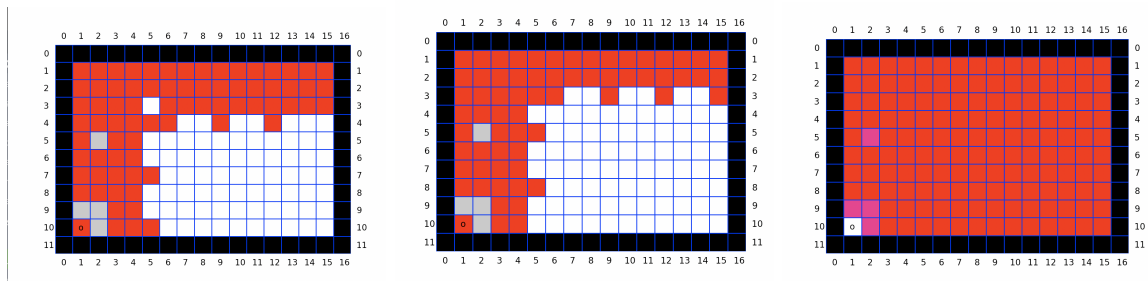


Figure 3.4: Exploration map left: cost-switch with button priority, middle: two part, right: regular manhattan

Table 3.1 shows the significant impact of the different cost metric types. Adding button prioritization and breaking the problem into subproblems by far resulted in the most significant improvements. This can be seen pretty clearly in the figure below, which shows that regular manhattan covers the whole board, while the more efficient methods do not cover the part of the board without important features.

Chapter 4

Conclusion

4.0.1 Conclusion

Through this project, we implemented the popular game bloxorz and developed both A-star and Dijkstra's in order to find paths from the start to the goal. We hoped to develop cost-metrics within this non-intuitive framework to allow the path to be found faster. Button prioritization was perhaps the most successful. By reordering the enqueue list by cost and placing all nodes that have seen a button at the front, we were able to drastically reduce run time. Other methods such as breaking the planning up into two parts and wall repelling were interesting to consider in the context of homework in the class. Breaking up the planning works similarly to starting from the goal and working back and meeting in the middle. Both are able to tackle the problem of specific targets as backworks planning help to go through narrow passagesways that are hard to find in one direction, while breaking the planning allows two more directed planners to work more efficiently. Wall repelling seems counterintuitive as typically walls are important to be near to discover small paths and entrances, however with this game, having a larger number of children tends to provide better results as nodes near walls usually end up getting stuck and having to move back. Delving into this topic allowed us to have fun with a cool application while learning about the intricacies of adjusting planners.

4.0.2 Project Takeaways

It was very interesting to see how varying the cost metrics changed variables such as nodes Done, run time, and total path length. If we did this project again, we likely would think a bit more about the underlying class structures that we used, and which attributes we assigned to each

class. For example, we had to eventually create an attribute called spaces for the nodes, which contained the spaces that were originally solely stored in the Maze class. This was because we implemented buttons and hidden tiles.

4.0.3 Special Thanks

Huge thank you to Professor Niemeyer and the TAs for all the help this term. We all learned a ton!

4.0.4 Link to Video

Bloxorz Video